Bank Saathi: AI Banking Companion

Product Requirements Document v2.1 (complete, merged)

Build4Better · 31 October 2026

Table of Contents

1. Summary

Bank Saathi is an Urdu-first web app (PWA) that helps people in Pakistan who did not grow up with technology, typically parents and grandparents, **learn** to use their own banking apps and understand their money, with confidence and without calling a family member every time.

The product follows one learning path: **1. Mashq (practise on a fake bank app) → 2. Saathi ke saath (do it on the real app with step-by-step guidance) → 3. Khud (do it alone).** It never moves money, never asks for a PIN, password or OTP, and is not a bank or a bank partner. The user performs every action in their own banking app.

Version 2.0 merges the original PRD (v1.0) with everything added during the build: Mashq practice mode, secure picture-code login with bank selection, bank guides for six banks, an official-vs-phishing contact checker, Zakat explanation, Fitra and Fidya calculator, in-app bill reminders, a full transparency log, and the Sauda (grocery) helper. **v2.1** moves all AI to Google Gemini (server-side), adds Urdu read-aloud for every Saathi reply, makes the assistant ask the user whenever something is unclear, and adds Google login with Firebase plus an encrypted Cloud Firestore backup.

2. Problem and target users

Primary user: bank customers in Pakistan who feel unsure using banking apps: older adults, first-time smartphone users, homemakers and shopkeepers who rely on a family member. Example persona (replace with real interview evidence, see `docs/USER_RESEARCH.md`): *Nasreen, 58, Karachi, has a bank app her son installed, calls him for every transfer, reads Urdu, partly understands English menu words, receives fake "BISP inaam" and "account blocked" SMS.*

Secondary users: adult children who act as unpaid tech support.

#	Problem	Addressed by
P1	Banking apps are intimidating; fear of pressing the wrong button	Mashq, Saathi assistant, bank guides
P2	Statements are unreadable;	Statement samjhein

#	Problem	Addressed by
	fees appear as codes	
P3	Cash withdrawals lose their purpose	Kharcha diary + reconciliation
P4	Scam calls, SMS and emails look real	Fraud check, Asli ya naqli contact checker, OTP-call rehearsal
P5	Zakat and Fitra are annual and error-prone	Zakat and Fitra calculators with explanations
P6	Bills and deadlines are missed	Bill yaad with in-app reminders
P7	Everyday money maths (grocery totals, change) is stressful	Sauda ka hisaab
P8	People don't know what they shared with an app	Meri history (transparency log)

3. Goals, non-goals, success metrics

Goals. G1: complete a real banking task alone after practice and guidance. G2: understand where last month's money went. G3: recognise a likely scam and know the safe next step. G4: feel more confident (measured, not assumed).

Non-goals. No money movement, no bank API, no collection of PINs, passwords, OTPs, CNICs or card numbers, no religious rulings (calculators explain and calculate only), no native app or wake word in v1.

Metric	Target
Task completion with assistant (2 tasks, 3–5 testers)	≥ 80%
Self-rated confidence (built-in 😟→😄 rating before/after each Mashq lesson)	rises ≥ 1 point
Scam verdicts on ~40 labelled messages	no known scam labelled "no red flags"; ≥ 90% agreement on clear cases
Statement parse integrity	100% of test statements, or clearly flagged
Ledger reconciliation match rate	≥ 90%
Reminder shown in-app for bills due ≤ 7 days	100% in demo

4. Product principles

1. **Teach, don't do.** Every step says *what* to tap and *why* ("Kyun?"); every task ends with a 3-line recap and a link back to practice.
2. **Never ask for secrets.** The assistant refuses PIN/OTP/password/card numbers and warns if a screenshot shows one.

3. **Be honest about uncertainty.** Unverified information is labelled; when unsure, send the user to the official helpline.
4. **Urdu first, voice friendly, forgiving.** Urdu (Nastaliq) and Roman Urdu toggle, 18px minimum text, "AA" big-text mode, voice input and read-aloud, mistakes explained, never punished.
5. **Minimum data, full transparency.** Images are never stored; everything that was said or shared is visible to the user in plain words.
6. **AI perceives and explains; code decides.** Matching, maths, rules, validation and calculations are deterministic and unit-tested.

5. Feature overview

#	Feature	Core technology	Status
F0	Mashq: practice on a fake bank app	Lesson data + simulator	Built
F1	Saathi: screenshot-guided assistant	OCR + Google Gemini vision (server) + task patterns	Built
F2	Statement samjhein (analyzer)	CSV/text/photo parsing, fee glossary	Built
F3	Kharcha diary + reconciliation	Deterministic matcher	Built
F4	Fraud check (messages)	Rule engine	Built
F5	Zakat calculator + plain explanation	Calculator	Built
F6	Bill yaad + in-app reminders	OCR extraction + reminders + .ics	Built
F7	Secure login + bank selection	Firebase Auth (Google) + WebCrypto PBKDF2 + AES-GCM	Built
F8	Mera bank: official contacts, guides, banking words	Verified directory	Built
F9	Asli ya naqli: number / email / website check	Directory + domain rules	Built
F10	Meri history: transparency log	Encrypted local log	Built
F11	Sauda ka hisaab (grocery helper)	Parser + OCR/vision	Built
F12	Fitra + Fidya calculator	Calculator with sourced yearly rates	Built

6. Feature requirements

F0. Mashq (practice mode)

A fake bank app ("Mashq Bank") inside a phone frame, clearly labelled *practice only, no real money*.

- Lessons: check balance (Available vs Ledger balance), send money to a saved beneficiary (look-alike names, amount entry, review screen, a fake "bank officer" call asking for the OTP mid-flow, typing the OTP only in the app), "Session expired / log in again", and an "asli ya naqli?" scam-message quiz.
- Every button carries a plain explanation; a wrong tap explains that button and points to the right one. After two misses the correct button is highlighted.
- Coach text in Urdu or Roman Urdu with read-aloud and a "Kyun?" line per step.
- Confidence rated 1–5 (faces) before and after each lesson and stored; completion recap; link to do it for real with Saathi.

F1. Saathi assistant (screenshot-guided)

Floating button on every screen. Text, voice and image input in one thread.

1. The user says the task (Urdu, Roman Urdu or English). The bank defaults to the one chosen after login.
2. The user logs in to their own app themselves; if a login screen is uploaded, Saathi says to type the password themselves and does not proceed.
3. For each screenshot, Saathi returns **one** instruction (≤ 2 sentences) quoting the on-screen English label, plus a "Kyun?" explanation, and highlights the button on the screenshot using OCR coordinates.
4. Money steps get a verification checkpoint (check name and amount; never share the OTP).
5. Recovery: "galat tap", "main kho gaya", "peechay".
6. End: recap + offers to log the payment in the Kharcha diary or practise again.

Requirements: strict JSON output validated with zod and one retry; screenshot text treated as untrusted data; OTP visible in a screenshot triggers a warning and is not repeated; only labels actually read on the screen may be named (no invented buttons); when unsure, the bank's verified helpline is shown. Works offline in a deterministic mode when no AI key is configured.

AI, language and voice (v2.1). Saathi uses Google Gemini through the server only (`GEMINI_API_KEY` never reaches the phone). Every reply is in Urdu script plus Roman Urdu, whatever language the user typed. Replies are read aloud: Gemini TTS first, then the phone's Urdu voice, then a Hindi voice reading the Roman Urdu line (Urdu is not in Gemini TTS's published language list, Oct 2026). An auto-read toggle is remembered. **Ask, don't guess:** when the message, screenshot or parchi is unclear, the model must set `needs_clarification` and ask one short question with tap-able answer chips; it never invents a button or a number.

Bank coverage (HBL, UBL, MCB, Meezan, NBP, ABL). Banks do not publish button-by-button guides and apps change often, so task patterns list the label variants banking apps

commonly use (e.g. “Fund Transfer / Send Money / Transfer”). The assistant picks whichever variant it reads on the user’s screen; if none is found it says so.

F2. Statement samjhein

Input: CSV, pasted text, or a photo/screenshot (on-device OCR). Parse into transactions, integrity check (opening + credits − debits = closing; suspect rows highlighted), categorise (user corrections remembered), explain every fee/tax line from a glossary with source and last-verified date (unverified entries labelled; unknown fees say “ask your bank”), flag anomalies, grounded Q&A from parsed data only, mask account numbers to the last 4 digits, never store the original file.

F3. Kharcha diary + reconciliation

Quick add in under 10 seconds (amount, source, purpose chips, note), voice entry (“paanch hazaar ghar ka rashan”), debit-SMS paste, edit and delete. Month end: exact amount, ±2 days, compatible type, one-to-one, closest date wins, ties flagged; ATM fee lines attach to their withdrawal; manual override; “Rs X withdrawn, Rs Y explained (Z%)” headline; 2-tap “what was this for?”; printable report.

F4. Fraud check (messages)

Paste text or upload a screenshot. Rules: OTP/PIN/CNIC requests, urgency/threats, prize/BISP/refund bait, short and look-alike links (checked against verified bank domains), pay-to-personal-account, remote-access apps (AnyDesk etc.), “call this mobile number”, bank message from a 03xx number. Verdicts: *Likely scam / Suspicious, verify first / No red flags found, still verify through your own app*: never “safe”. Red flags highlighted in the message; safe next steps; “warn my family” share. Checks run on the phone.

F5. Zakat calculator

Cash, bank (prefill from statement), gold by karat, silver, stock, receivables, investments, short-term liabilities; gram/tola toggle; silver or gold nisab with explanation; price source and timestamp; printable breakdown. **New:** after the result, a plain-language explanation (“aap ke paas kul …, qarz minus …, nisab …, 2.5% yaani har 100 mein 2.50 …”) and an option to save the summary to history. Scholar-difference notice.

F6. Bill yaad (bills and deadlines)

Add by photo (on-device OCR extraction; the user confirms values; the photo is deleted) or manually. Due soon / overdue / paid; recurring bills roll forward; bill-spike insight; “mark as paid” offers a diary entry; .ics calendar export with 7-day, 2-day and due-date alarms; family followers. **New:** immediately after a bill is added, an in-app message says when it must be paid, and a reminder banner appears at the top of every screen for bills due within 7 days or overdue. Browser notifications when permitted. WhatsApp/email reminders need a server (out of scope for v2).

F7. Secure login + bank selection

- **Tasveer Code:** 4 pictures out of 9, in order. Pictures (not digits) so users never re-use their bank PIN.
- The code derives an AES-256-GCM key with PBKDF2-SHA256 (600,000 iterations, random salt). All personal data is encrypted on the device; the key exists only in memory.
- Auto-lock after 5 minutes idle and on page reload; manual lock button; lockout after 5 wrong attempts (30 s, doubling, max 15 min).
- "Code bhool gaye?" explains that data cannot be recovered and offers a fresh start (deletes app data only; the bank account is unaffected).
- **Google login (Firebase Auth)** when the team's Firebase project is configured; the Tasveer Code is still needed on every open, so a stolen Google session alone does not open the data. Without Firebase the app runs device-only.
- After first login: name (no CNIC/account details) and **"Aap kaunsi bank app istemal karte hain?"** (HBL, UBL, MCB, Meezan, NBP, ABL, or "koi aur").

F8. Mera bank

Bank choice; official helplines (tap to call), the numbers the bank says it calls from, official website/email domains and verified emails, app name, the official source URL and verification date, and an explicit list of what could not be verified; task guides (F1 patterns with "Kyun?"); a searchable glossary of banking words in plain Urdu (Available balance, IBFT, Raast, IBAN, OTP, MPIN, TPIN, Session expired…).

F9. Asli ya naqli (number / email / website check)

Classifies an input against the verified directory: *official*, *naqli (look-alike)*, *bank ka nahi (personal mobile / Gmail)*, or *hamari list mein nahi*. Exact domain-suffix matching (no substring tricks). Always reminds that caller ID and email senders can be spoofed: hang up and call the number on the back of the card.

F10. Meri history (transparency log)

Every message the user typed or spoke, every Saathi reply, every piece of information shared (bank choice, bills, sauda list, zakat/fitra summaries, diary entries) and security events (login, lock, failed attempts), in plain Roman Urdu with time and feature. **Images:** never stored; the log keeps only metadata: size, dimensions, blur boxes, OCR on device, sent to AI or not, and when deleted. Saathi screenshots disappear from the screen after 2 minutes; other images immediately after reading. Filter, download and delete.

F11. Sauda ka hisaab (grocery helper)

- Input by typing, voice, or a photo of a handwritten parchi (vision AI if configured, else on-device OCR).
- Parses Roman/Urdu quantities and units (aadha kilo, pao, darjan, 500g, @ rate, written price).

- **Asks the user** whenever a line is unreadable, the quantity is missing, or the rate is unknown; remembers rates for next time.
- Shows each line's working (e.g. "500 gram = 0.5 kg × Rs 320 = Rs 160"), step-by-step addition, the total, change to receive from the amount paid, and the change broken into notes. Adds the total to the Kharcha diary.

F12. Fitra + Fidya calculator

Household members (adults and children), commodity choice with **2026 (1447 AH) per-person rates announced by the Council of Islamic Ideology on 9 Feb 2026** (wheat Rs 300 minimum, barley Rs 1,100, dates Rs 1,600, raisins Rs 3,800, munaqqa Rs 5,400; Fidya for 30 days: Rs 9,000 / 33,000 / 48,000 / 114,000 / 162,000), or a local rate; working shown ("5 afraad × Rs 300 = Rs 1,500"); plain explanation (who it is due for, pay before Eid prayer, give to the deserving); Fidya per missed fast; sources listed with a note that rates were taken from press reports of the CII announcement and change every year.

7. Language, accessibility and design

Urdu (Nastaliq) / Roman Urdu toggle everywhere; 18px minimum; "AA" big-text mode; 48px+ touch targets; five-item bottom bar on mobile (Ghar, Saathi, Mashq, Sauda, Fraud check) with every feature also a big tile on the home screen; plain words instead of jargon; voice input (Web Speech API) and read-aloud; one main action per screen; colour theme fixed by the team (primary #0B7A4B on white, danger #D93F3F, warning #C77B12).

8. Architecture and tech stack

Next.js 14 (App Router, TypeScript), Tailwind CSS, zod, Tesseract.js (self-hosted WASM + English data, OCR on the phone), Recharts (lazy-loaded), WebCrypto, Vitest (49 unit tests). Google Gemini (REST, server-side) for vision, handwriting and Urdu TTS; Firebase Auth (Google sign-in) and Cloud Firestore (encrypted backup), with the Firebase SDK lazy-loaded; jose for server-side Firebase ID-token checks. Server routes: `/api/assistant` (vision step), `/api/parchi` (handwriting transcription), `/api/tts` (Urdu speech); each requires a signed-in user when Firebase is configured and falls back to non-AI behaviour without a Gemini key.

Database choice: Cloud Firestore. Same project as login, per-user security rules, offline cache, free tier enough for a pilot, no server to run. Realtime Database offers nothing extra here; Supabase would add a second vendor and login system.

Module	Responsibility
`lib/practice`	Lessons, scam quiz
`lib/assistant`	Engine, task patterns, schema
`lib/statements`	Parser, categoriser, glossary, integrity
`lib/ledger`	Matcher, quick parse
`lib/scam`	Message rule engine

Module	Responsibility
`lib/banks`	Verified directory, contact check, terms
`lib/zakat`, `lib/fitra`	Calculators and sourced rates
`lib/bills`	Extraction validator, reminders, .ics
`lib/sauda`	Grocery parser and calculator
`lib/storage`, `lib/activity`	Encrypted storage + cloud sync, transparency log
`lib/ai/gemini`, `lib/firebase`, `lib/server`	Gemini client, Firebase Auth/Firestore, token check + API guard

9. Security and privacy

- Encryption at rest (AES-256-GCM, PBKDF2 600k), in-memory key, auto-lock, lockout.
- Images never stored; metadata only; account/card/phone numbers (10+ digits) masked to last 4.
- CSP, HSTS, X-Frame-Options DENY, nosniff, strict Referrer-Policy and Permissions-Policy; API routes check same origin, a signed-in Firebase user (token signature, issuer, audience, expiry), body size (enforced while reading) and per-user rate limits; `no-store` on API responses.
- AI output validated against strict schemas; image/OCR/message text treated as data, never instructions.
- Consent screen before the first upload; export and delete all data; delete everything and start over.
- Cloud Firestore holds ciphertext only; rules (`firestore.rules`) let each user read/write only `users/{uid}`, validate field names, types and sizes, and close everything else. Delete in Settings removes the phone and cloud copies.
- Secrets: only the Gemini key is secret and lives in server env vars; Firebase web config values are public identifiers. No key is written in the code.
- **Known limits (stated honestly):** a 4-picture code has 6,561 combinations; the lockout protects the app screen, but someone holding both the Google account and the encrypted data could brute-force it offline. Production should add passkeys and a stronger code. Gemini and Firebase were not tested live during the build (they need the team's keys).

10. Data model (on device and in Firestore, encrypted)

`profile` (name, bank, createdAt) · `ledger` · `statement` (parsed transactions, no file) · `categoryCorrections` · `matchOverrides` · `bills` · `zakatInputs/Prices` · `fitra*` · `saudaRates` · `practice` (lessons done, confidence before/after) · `activity` (transparency log). Plain: `auth` (salt, verifier, failed attempts), `lang`, `bigText`, `imageConsent`. Firestore: `users/{uid}` = {salt, iterations, verifier, updatedAt}; `users/{uid}/vault/{key}` = {ct (ciphertext), updatedAt}. The cloud copy wins when the app opens; keys only on the phone are uploaded.

11. Sources and verification

Item	Source (official unless noted)	Checked
HBL helplines, caller IDs 14250/14251, SMS 4250, customer.care@hbl.com	hbl.com/contact-us	31 Oct 2026
UBL 111-825-888, customer.services@ubl.com.pk	ubldigital.com/More/ContactUs	31 Oct 2026
MCB 111-000-622	mcb.com.pk (personal services call center)	31 Oct 2026
Meezan 111-331-331 / 111-331-332	meezanbank.com (call center notice)	31 Oct 2026
NBP 111-627-627	nbp.com.pk/HelpLine	31 Oct 2026
ABL 042-111-225-225, calls from 1222	abl.com/services/customer-services	31 Oct 2026
Fitra/Fidya 2026	CII announcement, 9 Feb 2026, via press reports (aaj.tv, dailypakistan.com.pk)	31 Oct 2026

Not verified: some app store names, banks' alphanumeric SMS sender IDs (not published), several masked emails, the fee glossary entries, exact button labels per bank. The app labels these as unverified.

12. Non-functional requirements

Mobile-first and light: first load ≈ 90–110 KB JS per page, OCR and charts loaded only when used, works on a 390px screen without horizontal scroll. Screenshot to instruction ≈ 6 s typical. Every AI feature has a non-AI fallback. Unit tests for the matcher, zakat, fitra, scam rules, parser, bill extractor, sauda parser, contact checker, encryption, assistant engine and practice lessons.

13. Demo plan (2 minutes)

0:00 the user and the problem · 0:15 Mashq with the fake OTP call · 0:45 Saathi on a real screenshot (auto-blur, highlight, "Kyun?") · 1:10 Fraud check + Asli ya naqli · 1:30 Sauda parchi with a question and the working · 1:45 history page, confidence 😟→😄, honest limits. Full script: `docs/DEMO_SCRIPT.md`.

14. Risks and mitigations

Risk	Mitigation
Bank apps block screenshots	Photo with a second phone; type what is on screen; Mashq for learning
Wrong step on a money screen	Only labels read on screen, checkpoints, helpline fallback
Handwriting not readable	Saathi asks line by line; typing/voice fallback
Stale rates (zakat prices, fitra)	Source + date shown, manual override, yearly update
Weak picture code	Google login + code, lockout, encryption, honest disclosure, passkeys in roadmap
Urdu voice quality	Gemini TTS, then phone Urdu voice, then Hindi voice on Roman Urdu; text always shown
AI cost / abuse	Signed-in users only, rate limits, spending cap in AI Studio
Over-reassuring scam checks	Never “safe”; always the verification route

15. Roadmap

v2.2: passkeys; Firebase App Check; WhatsApp/email reminders; per-bank verified label packs built with real app walk-throughs; family helper mode. v3: native app with user-approved screen sharing; Sindhi, Pashto, Punjabi.